

MetaSAEs: Joint Training with a Decomposability Penalty Produces More Atomic Sparse Autoencoder Latents

Anonymous authors

Paper under double-blind review

Abstract

Sparse autoencoders (SAEs) are increasingly used for safety-relevant applications including alignment detection and model steering. These use cases require SAE latents to be as atomic as possible. Each latent should represent a single coherent concept drawn from a single underlying representational subspace. In practice, SAE latents blend representational subspaces together. A single feature can activate across semantically distinct contexts that share no true common representation, muddying an already complex picture of model computation. We introduce a joint training objective that directly penalizes this subspace blending. A small *meta* SAE is trained alongside the primary SAE to sparsely reconstruct the primary SAE’s decoder columns; the primary SAE is penalized whenever its decoder directions are easy to reconstruct from the meta dictionary. This occurs whenever latent directions lie in a subspace spanned by other primary directions. This creates gradient pressure toward more mutually independent decoder directions that resist sparse meta-compression.

On GPT-2 large (layer 20), the selected configuration reduces mean $|\phi|$ by 7.5% relative to an identical solo SAE trained on the same data. Automated interpretability (fuzzing) scores improve by 7.6%, providing external validation of the atomicity gain independent of the training and co-occurrence metrics. Reconstruction overhead is modest. The joint SAE increases CE loss by 3.1% above the baseline language model. This compares to 2.5% for the solo SAE alone, an additional overhead of 0.6 percentage points. Results on Gemma 2 9B are directional. On not-fully-converged SAEs, the same parameterization yields the best results, a +8.6% ΔFuzz . Though directional, this is an encouraging sign that the method transfers to a larger model. Qualitative analysis confirms that features firing on polysemantic tokens are split into semantically distinct sub-features, each specializing in a distinct representational subspace.

1 Introduction

Mechanistic interpretability aims to understand the computations of neural networks in terms of human-interpretable representations and computations. Sparse autoencoders (SAEs) have become a central tool for this effort, learning overcomplete dictionaries of features that decompose dense model activations into sparse, hopefully monosemantic latents (Bricken et al., 2023; Cunningham et al., 2023; Templeton et al., 2024). Beyond basic interpretability research, SAE features are increasingly used in practical applications: detecting alignment risk, and steering model behavior by intervening in the latent space. These downstream uses place a premium on *atomic* latents, features that each represent one semantic feature rather than a blend of several.

The non-atomicity problem. Neural network representations are distributed across many overlapping subspaces (Elhage et al., 2022). An ideal SAE latent would specialize to a single such subspace, making the decomposition clean and interpretable. In practice, SAE training does not guarantee this. A single latent can blend representations from distinct semantic subspaces, activating on multiple unrelated contexts that share no true common

underlying representation. This *subspace blending* muddies an already complex picture of model computation. [Bussmann et al. \(2024\)](#) showed this concretely. By training a secondary (“meta”) SAE to reconstruct the decoder columns of a primary SAE, they demonstrated that primary latents routinely decompose into combinations of meta-latents. A single “Einstein” latent, for example, decomposed into “scientist”, “German”, and “famous person” meta-latents. The primary SAE had merged conceptually distinct representational directions into one feature. In this particular case, Einstein may still be a good use of a dictionary element. This demonstrates the point that an SAE may blend features together that confound important use cases. For example, a feature conflating two concepts that is activated when steering a model will encourage the desired behavior while also propagating effects in other, semantically distinct contexts.

Our approach. Rather than diagnosing non-atomicity post-hoc, can we reduce it during training? We introduce a joint training objective in which the meta SAE is trained *simultaneously* with the primary SAE. The meta SAE continuously tries to compactly reconstruct the primary decoder columns. The primary SAE is penalized when its decoder vectors lie in low-dimensional subspaces already captured by the meta dictionary, making them easy to reconstruct. This creates a gradient pressure that pushes primary features into regions of activation space that resist sparse meta-reconstruction, driving the primary dictionary toward greater mutual independence.

Contributions.

- A joint training objective combining primary and meta SAE training with a decomposability penalty (§3).
- Evidence that joint training increases feature atomicity, measured via pairwise co-occurrence rates ($|\phi|$) and automated interpretability scoring (§5), with careful consideration of alternative explanations (§7.1).
- Qualitative case studies demonstrating semantically distinct feature specialization (§5.4).
- Positive directional results on Gemma 2 9B (§6) with the same winning parameterization as GPT-2 large.

2 Background

Sparse autoencoders. An SAE learns an encoder $f_{\text{enc}} : \mathbb{R}^d \rightarrow \mathbb{R}^n$ and decoder $f_{\text{dec}} : \mathbb{R}^n \rightarrow \mathbb{R}^d$ with $n \gg d$ and a sparse latent code. We use BatchTopK SAEs ([Gao et al., 2024](#)). Given a batch of activations, the top- k pre-activations across *all* latents in the batch are retained, enforcing an exact batch-level L0 of k . The i -th decoder column $W_{\text{dec}}[i] \in \mathbb{R}^d$ is the “feature direction” for latent i . SAEs have been applied to GPT-2 ([Cunningham et al., 2023](#)), Claude models ([Templeton et al., 2024](#)), and Gemma ([Lieberum et al., 2024](#)), and have been used to identify safety-relevant internal features, including ones associated with deception, bias, sycophancy, and dangerous content ([Templeton et al., 2024](#)).

Feature non-atomicity and superposition. [Elhage et al. \(2022\)](#) argued that neural networks represent more features than they have dimensions by encoding them as directions in an overcomplete basis. This superposition hypothesis implies that a single SAE latent may be pulled toward directions that blend multiple underlying features, particularly when the training objective (reconstruction fidelity plus sparsity) does not directly penalize such blending. [Bussmann et al. \(2024\)](#) introduced meta SAEs: a secondary SAE trained on the decoder columns of a primary SAE, treating each column as an input vector. A small meta dictionary ($m \ll n$) is forced to find shared structure across primary features. Low meta-reconstruction error for primary feature i indicates that $W_{\text{dec}}[i]$ lies near the subspace spanned by the current meta dictionary. In their setup the meta SAE is trained sequentially on a frozen primary, so no signal flows back to improve it.

Orthogonality penalties. Korznikov et al. (2025) (OrtSAE) pursue a related goal via a direct orthogonality penalty on primary SAE decoder columns, penalizing high pairwise cosine similarity within chunks of the decoder matrix. OrtSAE and our method share the motivation of pushing feature directions apart, but differ in approach. OrtSAE enforces pairwise geometric separation uniformly across all decoder column pairs, whereas our decomposability penalty targets *sparse reconstructability*. This distinction means our penalty is more sensitive to the semantic structure of the feature space, rather than treating all pairs symmetrically.

Automated interpretability and fuzzing. Bills et al. (2023) introduced the use of language models to generate natural-language descriptions of neural network features and evaluate their quality. Paulo et al. (2024) extended this to the *fuzzing* paradigm. Given a candidate description of a feature, a judge model must identify which of two activation examples is real versus randomly modified (highest-activating tokens replaced by random draws from the same marginal distribution). Scores range from -1 (consistently wrong) to $+1$ (always correct), with 0 at chance. Fuzzing is particularly sensitive to atomicity. A blended feature activating on two unrelated concepts cannot be coherently described in a way that reliably predicts which examples are real. So blended features tend to score poorly compared to single-concept features.

The phi coefficient as a co-occurrence metric. For two binary firing indicators (latent i fires / does not fire, latent j fires / does not fire), the phi (φ) coefficient measures co-occurrence above chance:

$$\varphi(i, j) = \frac{N_{11}n - N_{1\cdot i}N_{\cdot 1j}}{\sqrt{N_{1\cdot i}(n - N_{1\cdot i})N_{\cdot 1j}(n - N_{\cdot 1j})}}, \quad (1)$$

where N_{11} is the number of tokens on which both latents fire, $N_{1\cdot i}$, $N_{\cdot 1j}$ are marginal firing counts, and n is total tokens. We use mean $|\varphi|$ as our primary co-occurrence metric throughout. Crucially, φ is normalized by the marginal firing rates: changes in how often individual features fire do not by themselves change φ .

Why mean $|\varphi|$ as a grid selection criterion. One might ask whether reducing mean $|\varphi|$ is a meaningful goal: splitting a polysemantic solo feature into two joint features could increase or decrease the total co-occurrence depending on the specifics of the split. We argue that the dominant type of co-occurrence in a large overcomplete dictionary is *subtle subspace blending*. Many feature pairs have small but non-zero $|\varphi|$. Subspaces are represented dissimilarly but not fully independently. If the decomposability penalty succeeds in pushing decoder directions toward greater mutual independence, this mass of small non-zero correlations should decrease, and the numerical dominance of these many pairs in the mean will reliably drive mean $|\varphi|$ down even if a small number of well-separated “big-star” splitting candidates move in either direction. Fuzzing scores serve as independent external validation of this claim (§5.2).

3 Method

3.1 Architecture

We train a *primary SAE* on model activations and a *meta SAE* on the primary SAE’s decoder columns simultaneously. Both use BatchTopK. The primary SAE has dictionary size n . The meta SAE has dictionary size $m \ll n$. We follow Bussmann et al. (2024) in using a compression ratio of $m/n \approx 9\%$ (1,800 meta-features for a 20,480-feature primary dictionary; their ratio was $\approx 5\%$). With $m \ll n$, the meta SAE cannot memorize individual decoder columns. It must find directions that are reused across the primary dictionary.

3.2 Decomposability penalty

Let $\hat{W}_{\text{dec}}[i]$ be the meta SAE’s reconstruction of primary decoder column i . We define a per-feature penalty that is large when reconstruction error is small:

$$p_i = \exp\left(-\frac{\|W_{\text{dec}}[i] - \hat{W}_{\text{dec}}[i]\|^2}{\sigma^2}\right), \quad (2)$$

and augment the primary SAE’s loss:

$$\mathcal{L}_{\text{primary}} = \mathcal{L}_{\text{SAE}}(x) + \lambda_2 \cdot \frac{1}{n} \sum_{i=1}^n p_i. \quad (3)$$

$\mathcal{L}_{\text{SAE}}$ is the standard L2 reconstruction loss plus an auxiliary dead-feature loss. λ_2 controls the strength of the penalty; σ^2 is a bandwidth parameter that determines how sharply the penalty distinguishes low- from high-error reconstructions.

Intuitively: $p_i \approx 1$ when the meta SAE reconstructs feature i easily (its decoder direction lies in the meta-feature subspace). $p_i \approx 0$ when feature i is genuinely novel relative to the meta dictionary. Minimizing $\sum_i p_i$ pushes the primary dictionary toward features that are more mutually independent and resist sparse meta-compression.

3.3 Training procedure and gradient flow

Training alternates at each macro-step:

1. **Primary phase** (n_p steps): compute the primary SAE forward pass and the decomposability penalty, treating the current meta SAE as a frozen critic; update primary SAE parameters.
2. **Meta phase** (n_m steps): use the current primary decoder matrix W_{dec} as training data; update the meta SAE to minimize its reconstruction loss on these vectors.

In Phase 1, $\hat{W}_{\text{dec}}[i]$ is computed inside `torch.no_grad()` so gradients flow to W_{dec} but not through the meta SAE; the resulting gradient points each decoder column away from its current meta-reconstruction, into regions the meta dictionary has not yet learned. As the primary decoder updates, the meta SAE adapts in Phase 2, creating an adversarial dynamic akin to diversity regularization in ensemble methods (Krogh & Vedelsby, 1995), but operating on the geometry of the feature dictionary. The result is a set of primary features that are more mutually independent: each occupies a direction not predictable from the others, reducing the subspace blending that underlies non-atomicity.

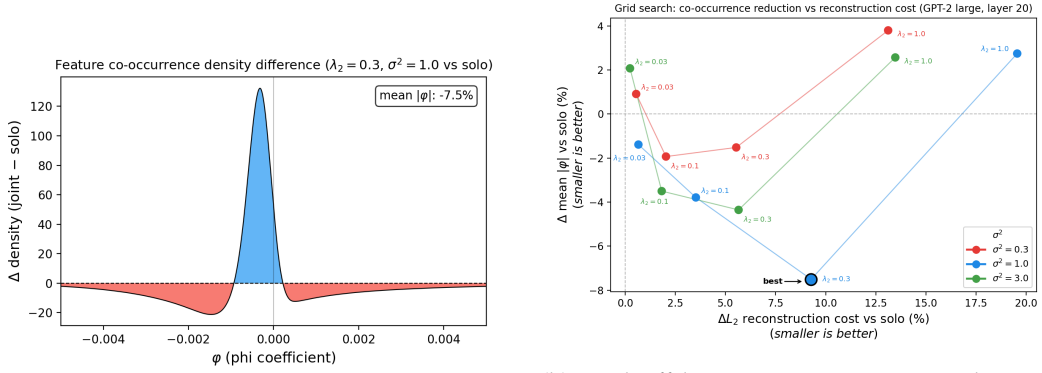
4 Experimental setup

Model and layer. We train SAEs on residual stream activations at layer 20 (`resid.pre`) of GPT-2 large (Radford et al., 2019) ($d = 1280$, 36 layers). Generalization experiments use Gemma 2 9B (Gemma Team, 2024) at layer 23 ($d = 3584$).

Dataset. FineWeb 10BT (Penedo et al., 2024), streamed with context length 128. SAE training uses 100M tokens; co-occurrence evaluation uses 35M fresh tokens.

SAE architecture. Primary SAE: $n = 20,480$ features, batch $k = 64$. Meta SAE: $m = 1,800$ features, batch $k = 8$. We chose BatchTopK (Gao et al., 2024) over JumpReLU (Rajamanoharan et al., 2024) via an empirical ablation comparing both architectures across 8 configurations. BatchTopK achieved lower L2 loss in both joint and solo conditions and was used for all subsequent experiments.

Hyperparameter sweep. We sweep $\lambda_2 \in \{0.1, 0.3, 1.0\}$ and $\sigma^2 \in \{0.3, 1.0, 3.0\}$ (9 configurations), each trained for 100M tokens with $n_p = 10$ primary steps and $n_m = 5$ meta steps



(a) Histogram difference (joint – solo) in $|\varphi|$ for the best configuration ($\lambda_2 = 0.3$, $\sigma^2 = 1.0$). Negative bars indicate probability mass shifted away from correlated co-activation

(b) Tradeoff between co-occurrence reduction ($\Delta|\varphi|$) and reconstruction cost (ΔL_2) across all 12 grid configurations. $\lambda_2 = 1.0$ (red) achieves poor tradeoffs; the best Pareto point is $\lambda_2 = 0.3$, $\sigma^2 = 1.0$.

Figure 1: Co-occurrence results on GPT-2 large, layer 20.

per macro-step, Adam with $\text{lr} = 3 \times 10^{-4}$. The solo baseline is an identically configured BatchTopK SAE trained without the meta SAE or penalty. We select the best configuration on mean $|\varphi|$ from the 35M-token co-occurrence scan while considering reconstruction quality. Automated interpretability scores serve as independent validation of the selected configuration (§5.2).

Co-occurrence evaluation. Per-feature firing thresholds are calibrated using an equal-error-rate (EER) procedure on 5M tokens (Appendix A), giving reproducible per-token binary firing decisions at inference time independent of batch composition.

Automated interpretability. We compute fuzzing scores (Bills et al., 2023; Paulo et al., 2024) using Qwen2.5-72B-Instruct-AWQ as the judge model. We align joint and solo features via Hungarian matching on decoder cosine similarity, yielding 20,456 matched pairs, and report paired Δ -scores.

5 Results

5.1 Feature co-occurrence

In Table 1 we report mean $|\varphi|$ and L2 across all 12 grid configurations. The best configuration ($\lambda_2 = 0.3$, $\sigma^2 = 1.0$) achieves a 7.5% reduction in mean $|\varphi|$, with +9.3% L2 increase. The effect is highly significant. Welch’s t -test over all $\approx 209\text{M}$ pairs gives $t = -53.1$, $p \ll 10^{-100}$.

We verified all metrics for a $\lambda_2 = 0$ control (joint architecture, no penalty), confirming no systematic effect on $|\varphi|$, L2, or fuzzing scores relative to the solo baseline. The effect is therefore attributable to the decomposability penalty itself, not to the joint optimization structure. $\lambda_2 = 1.0$ degrades reconstruction severely without commensurate $|\varphi|$ gains. Over-penalizing decoder reconstruction forces directions apart even when shared geometry reflects genuine co-structure in the data, not superficial blending. The Pareto-optimal region, $\lambda_2 \in \{0.1, 0.3\}$, offers the best $|\varphi|$ reduction per unit reconstruction cost (Figure 1b).

Independent baseline. Four independent solo SAEs yield mean $|\varphi| \approx 2.15 \times 10^{-3} \pm 10^{-5}$ —natural run-to-run variation far smaller than the 1.7×10^{-4} joint-solo gap, confirming the effect is specific to joint training rather than stochastic optimization noise.

λ_2	σ^2	$\Delta \varphi $ (%)	ΔL2 (%)	ΔCE (%)
0.1	0.3	-1.9	+2.0	—
0.1	1.0	-3.8	+3.5	+2.8
0.1	3.0	-3.5	+1.8	+2.7
0.3	0.3	-1.5	+5.5	—
0.3	1.0	-7.5	+9.3	+3.1
0.3	3.0	-4.4	+5.7	+2.9
1.0	0.3	+3.8	+13.1	—
1.0	1.0	+2.8	+19.6	+3.5
1.0	3.0	+2.6	+13.5	+3.2

Table 1: Co-occurrence and reconstruction across the hyperparameter grid, relative to the solo baseline ($|\varphi| = 2.27 \times 10^{-3}$, L2= 0.097). ΔCE is joint CE overhead as % of baseline language-model CE (3.47 nats); solo overhead is 2.4%. Configs without CE evaluation marked “—”. Best row **bold**.

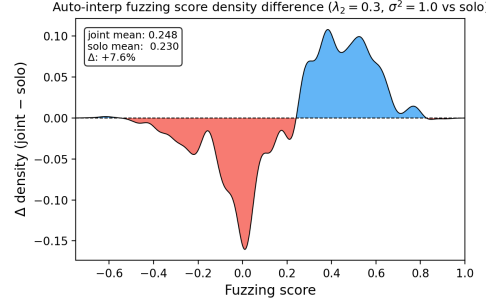


Figure 2: Density difference (joint – solo) in fuzzing score distributions. Blue: higher density in joint; red: higher density in solo. Joint training shifts mass from near-zero scores toward higher values, indicating more consistently interpretable features.

5.2 Automated interpretability

Mean fuzzing score on the jointly trained primary SAE is 0.248, compared to 0.230 for the solo SAE (paired $\Delta = +0.0175$, +7.6%; $t = 8.12$, $p = 5 \times 10^{-16}$); 51.6% of pairs show higher joint scores (Figure 2). As argued in §2, this constitutes external validation of the atomicity claim independent of the co-occurrence analysis.

5.3 Reconstruction quality

For the best configuration ($\lambda_2 = 0.3, \sigma^2 = 1.0$):

- **CE overhead:** the solo SAE increases CE loss by 2.5% above the baseline language model (absolute increase of 0.087 nats over baseline CE of 3.47). The joint SAE increases CE by 3.1%, an additional overhead of 0.6 percentage points above the solo.
- **L2 loss:** +9.3% relative increase; absolute L2 increase = 0.010.
- **Absolute scale:** mean activation norm $\|x\|_2 = 133.3$ at this layer; the additional L2 cost is 0.0075% of mean norm—negligible in absolute terms.
- **L0:** 64 for both conditions, enforced by BatchTopK.

For reconstruction-sensitive applications, $\lambda_2 = 0.1, \sigma^2 = 3.0$ offers a better tradeoff: -3.5% $\Delta|\varphi|$ at only +1.8% ΔL2 .

5.4 Qualitative case studies (GPT-2 large)

The *cross- $|\varphi|$ matrix* records $|\varphi|$ between every solo feature and every joint feature (co-occurrence correlations across the two SAEs rather than within one), and identifies *splitting candidates*. These are solo features that co-activate strongly with multiple semantically distinct joint features, suggesting the solo SAE blended subspaces that the joint SAE represents as separate directions. We inspect splitting candidates from the $\lambda_2 = 0.3, \sigma^2 = 1.0$ GPT-2 large pipeline. Full feature descriptions and scores are in Tables 2 and 3.

Two cases illustrate the pattern. First, the English word *too* is polysemantic. It can mean “excessively” (degree modifier, e.g. *it was too hot*) or “also / additionally” (sentence-final, e.g. *me too*). The solo SAE conflates both senses in a single feature (fuzz +0.727), blending two distinct semantic subspaces. The joint SAE resolves them: feat. 4741 fires on the excess/degree-modifier use in contexts of regret or negative outcome (fuzz +0.681, $|\varphi| = 0.710$), while feat. 14213 captures the sentence-final additive use (fuzz +0.174, $|\varphi| = 0.799$).

Second, the word *let* serves at least three distinct pragmatic roles: direct suggestion or imperative (“Let’s go”), discourse-transition pivot (“Let me now turn to...”), and permissive allowing. The solo SAE merges all three into one feature (fuzz +0.641), conflating subspaces with different pragmatic and syntactic properties. The joint SAE assigns each role to a distinct feature: feat. 7228 takes the directive/invitation use ($|\varphi| = 0.692$, fuzz +0.646), feat. 10152 the discourse-pivot use ($|\varphi| = 0.486$, fuzz +0.273), and feat. 18650 the permissive-allowing use ($|\varphi| = 0.435$, fuzz +0.497). Both cases are consistent with the mutual independence hypothesis. The decomposability penalty encourages the joint SAE to disentangle uses of the same surface form that occupy distinct representational subspaces.

6 Generalization to Gemma 2 9B

We apply the joint training method to Gemma 2 9B (Gemma Team, 2024) (layer 23, $n = 65,536$ features, $L_0 = 100$) sweeping $\lambda_2 \in \{0.1, 0.3, 1.0\}$ at $\sigma^2 \in \{1.0, 3.0\}$ on 75M FineWeb tokens. We emphasize that these are well-optimized but *not fully converged* SAEs, and the grid co-occurrence scan uses a smaller token budget (5M tokens over a $65,536^2$ feature space), giving wide confidence intervals. Gemma results should be interpreted as directional.

Config selection. The co-occurrence grid on Gemma is less conclusive than on GPT-2 large due to the underpowered scan; $|\varphi|$ effects are small and mixed across configurations. Notably, $\lambda_2 = 1.0$ shows the same failure mode as on GPT-2 large—substantial $+|\varphi|$ increases alongside heavy L2 overhead—confirming this regime is problematic regardless of model scale. We select $\lambda_2 = 0.3$, $\sigma^2 = 1.0$ as the primary Gemma configuration, identical to the GPT-2 large winner. Encouragingly, this is the only Gemma configuration that shows meaningful positive automated interpretability gains (§6 below).

Reconstruction overhead. The L2 overhead pattern on Gemma is consistent with GPT-2 large and scales with λ_2 : +8.2% at $\lambda_2 = 0.3$, $\sigma^2 = 1.0$ (compared to +9.3% on GPT-2 large at the same setting). For the selected parameters, we continued training from the grid scan checkpoint for an additional 300M tokens. This better trained joint SAE adds a modest additional overhead of 0.0196 CE units (+2.5%) above the solo SAE’s 0.1602 CE increase (+2.2%) over baseline (7.12 nats).

Automated interpretability. On the grid fuzz sweep ($n \approx 1,000$ random features per configuration), $\lambda_2 = 0.3$, $\sigma^2 = 1.0$ is the only Gemma configuration with clearly positive ΔFuzz at +8.6%, directionally consistent with the GPT-2 large result. The fact that the best Gemma configuration matches the GPT-2 large winner without any Gemma-specific tuning is encouraging evidence that $\lambda_2 \approx 0.3$, $\sigma^2 \approx 1.0$ may be a robust default.

Qualitative examples. We identify splitting candidates from the Gemma 2 9B cross- $|\varphi|$ scan using the pipeline described in 5.4. See details in Tables 4 and 5 in the appendix.

The word *cookies* illustrates a clean two-way sense split. The solo SAE conflates the food sense (*baked cookies*) and the web-technology sense (*browser cookies, data tracking*) in a single feature (fuzz +0.493). The joint SAE resolves them: feat. 7892 specializes on food/baking (fuzz +0.548, $|\varphi| = 0.741$), while feats. 29453 and 34623 each cover the web/privacy sense from slightly different angles ($|\varphi| = 0.875$ and 0.621).

The word *flash* illustrates the liberation of a rare but highly coherent sense. The solo feature covers the general sense of a sudden, brief occurrence (fuzz +0.602). The joint SAE splits out feat. 34102, which specializes entirely on Adobe Flash software (fuzz +0.725, $|\varphi| = 0.173$). The low $|\varphi|$ reflects how rarely this sense fires relative to the broader usage. But its fuzzing score is the highest of any feature in the Gemma analysis, confirming it is a precise semantic unit that the solo SAE had submerged within a broader feature.

7 Discussion

7.1 Do the results mean what we claim?

We claim joint training produces more *atomic* latents, supported by two independent measures: co-occurrence rates and automated interpretability scores. We address the most natural alternative explanations.

Lower firing rates? ϕ is normalized by marginal firing rates, so firing-rate changes alone cannot move it. BatchTopK further enforces *identical* batch-level L0 = 64 for both conditions, fixing the mean per-feature rate exactly. Appendix B shows the full firing-rate distributions are comparable, ruling out distributional differences as an explanation.

Near-dead features? Dead features trivially contribute $\phi \approx 0$. The best joint configuration for GPT-2 large has *zero* dead features (fewer than the 2 in the solo baseline), and Appendix B confirms no long tail of near-zero-activity features.

Specialization rather than atomicity? A narrower but coherent feature would also score higher on fuzzing. But genuine splitting makes a distinct structural prediction: solo features should exhibit high co-occurrence with *multiple* distinct joint features. The qualitative case studies (§5.4) test this directly and confirm the expected multi-way split structure.

Better reconstruction driving auto-interp? No: joint reconstruction is *worse* (L2 +9.3%, CE overhead +0.6pp above solo), so the evaluator cannot be benefiting from higher fidelity.

Joint architecture without the penalty? The $\lambda_2 = 0$ controls show no systematic reduction. The effect scales monotonically with λ_2 , confirming the penalty, not the architecture, is the driver.

7.2 Limitations and future work

Hyperparameters. The penalty adds λ_2 and σ^2 . Our grid suggests $\lambda_2 \in [0.1, 0.3]$, $\sigma^2 \approx 1.0$ as practical defaults, with $\lambda_2 = 0.1$ offering a better reconstruction tradeoff for applications where CE overhead is a primary concern. The consistency of this finding across GPT-2 large and Gemma 2 9B is encouraging, though further confirmation at additional scales is needed.

Meta dictionary size. We follow Bussmann et al. (2024) in setting $m/n \approx 9\%$ without ablation. Understanding how m affects the compression pressure is an important direction for future work.

Scope. We evaluate at one layer per model. How atomicity gains vary with layer depth, architecture, and model scale remain open questions. The decomposability penalty targets decoder-direction mutual independence. It does not directly address all sources of feature noise or non-interpretability in SAEs.

8 Conclusion

We presented a joint training method that directly incentivizes SAE latents toward greater atomicity by penalizing subspace blending among decoder directions. A meta SAE trained alongside the primary SAE acts as a continuous critic, penalizing primary features whose decoder directions lie in the low-dimensional subspace already captured by the meta dictionary. On GPT-2 large, this reduces mean $|\phi|$ by 7.5% and improves automated interpretability scores by 7.6% ($p = 5 \times 10^{-16}$), at a modest additional CE overhead of 0.6 percentage points above the solo SAE. Directional results on Gemma 2 9B (+8.6% Δ Fuzz) suggest the method transfers across model scales with the same hyperparameter setting. These gains hold up under scrutiny of the most plausible alternative explanations, and qualitative case studies

confirm that the $|\varphi|$ reduction corresponds to genuine semantic splitting of features that blend distinct representational subspaces. The method is simple, architecture-agnostic, and adds no data requirements beyond standard SAE training; we hope it proves useful for applications that require reliable, atomic feature representations.

9 Reproducibility and LLM Disclosure

The code is available at <https://github.com/good-epic/MetaSAE>. Submit issues on github or email the author with questions or problems.

The author leveraged LLMs in the following ways to complete this work:

- *Literature review and background research*
- *Learning new concepts*. For example, I was not familiar with optimal transport theory before this project and used both reading and Q&A with LLMs to learn about it.
- *Programming assistance*. I wrote inline with Cursor and leveraged Claude Code to write code from scratch after extensive specification. I reviewed all code to ensure it was correct and functional.
- *Drafting the paper*. I interactively developed an outline and first draft with Claude Code, then extensively edited and revised. I subsequently asked for skeptical rereads as well.

References

- Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models. *OpenAI Blog*, 2023. URL <https://openaireview.io/forum?id=58bzP6t08T>.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Bart Bussmann, Michael Pearce, Patrick Leask, Joseph Bloom, Lee Sharkey, and Neel Nanda. Showing SAE latents are not atomic using meta-SAEs. *LessWrong*, 2024. URL <https://www.lesswrong.com/posts/TMAmHh4DdMr4nCSr5>.
- Hoagy Cunningham, Aidan Ewart, Logan Sherborne, Tom Henighan, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.
- Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Tow, Alec Bas, Hoagy Cunningham, Tom Conerly, Tom Henighan, et al. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- Kirill Korznikov et al. OrtSAE: Orthogonal sparse autoencoders. *arXiv preprint arXiv:2509.22033*, 2025.
- Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems*, volume 7, 1995.

- 367 Tom Lieberum, Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat,
 368 János Kramár, Rohin Shah, Neel Nanda Beren, Mor Brikman, et al. Gemma scope: Open
 369 sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*,
 370 2024.
- 371 Gonçalo Paulo, Alex Mallen, Caden Juang, and Nora Belrose. Automatically interpreting
 372 millions of features in large language models. *arXiv preprint arXiv:2410.13928*, 2024.
- 373 Guilherme Penedo, Hynek Kydlíček, Javier de la Rosa, Anton Lozhkov, Margaret Mitchell,
 374 Thomas Wolf, Leandro Von Werra, and Matteo Cappelli. FineWeb: Decanting the Web for
 375 the finest text data at scale, 2024.
- 376 Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever.
 377 Language models are unsupervised multitask learners. In *OpenAI Blog*, 2019.
- 378 Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma,
 379 János Kramár, Rohin Shah, and Neel Nanda. Improving dictionary learning with gated
 380 SAEs. *arXiv preprint arXiv:2404.16014*, 2024.
- 381 Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen,
 382 Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, et al. Scaling monosemantic-
 383 ity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*,
 384 2024. URL [https://transformer-circuits.pub/2024/scaling-monosemanticity/index.](https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html)
 385 [html](https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html).

386 A Calibration of per-feature firing thresholds

387 Co-occurrence evaluation with BatchTopK SAEs poses a subtlety: the top- k selection is
 388 applied over an entire batch, so whether feature i fires on a given token depends on what
 389 other tokens are in the batch. This makes firing a batch-composition-dependent decision,
 390 which is unsuitable for accumulating reproducible per-token co-occurrence statistics.

391 We resolve this by calibrating a per-feature firing threshold θ_i using an equal-error-rate
 392 (EER) criterion on 5M tokens. For each feature i , we find θ_i such that the rate at which the
 393 feature fires above θ_i (under θ -thresholding) matches its BatchTopK firing rate as closely as
 394 possible. Co-occurrence statistics are then computed using these fixed thresholds: feature i
 395 fires on token t if and only if its pre-activation exceeds θ_i , independently of other tokens.
 396 This gives reproducible, batch-composition-independent binary firing indicators.

397 B Firing rate distributions

398 Per-feature firing rates for the joint ($\lambda_2 = 0.3, \sigma^2 = 1.0$) and solo SAEs were computed over
 399 35M tokens using the calibrated EER thresholds. Both distributions show closely matched
 400 spread and shape: mean firing rate per feature is identical by construction (BatchTopK with
 401 $L_0 = 64$ fixes the aggregate), and the per-feature distributions are comparable. No long tail
 402 of near-zero-activity features appears in the joint condition, ruling out near-dead features as
 403 a driver of the $|\varphi|$ reduction. Firing rate data are stored in the `N1_joint` and `N1_solo` vectors
 404 of `cross_phi_topk.npz`.

405 C $|\varphi|$ distribution shape

406 Approximately 75.7% of all ≈ 209 M feature pairs have $\varphi < 0$: most pairs are weakly anti-
 407 correlated by construction, since sparse activations in a large overcomplete dictionary cannot
 408 push many pairs simultaneously positive. The median $\varphi \approx -5 \times 10^{-4}$; the mean is slightly
 409 positive due to a heavy right tail reaching $\varphi \approx 0.13$ for the most strongly co-activating pairs.
 410 Joint training acts specifically on the positive tail (Figure 1a), redistributing mass toward
 411 zero without meaningfully altering the dominant anti-correlated region. This is consistent
 412 with the subspace-blending interpretation: the penalty acts on directions that share structure
 413 (positive φ), leaving uncorrelated or anti-correlated pairs unchanged.

Table 2: **“too” polysemy split.** Solo feature 2827 conflates the excess-degree sense (*it was too hot*) with the sentence-final additive sense (*me too; that’s true too*). The joint SAE separates them: feat. 4741 inherits the excess sense ($|\varphi| = 0.710$), while feat. 14213 captures the additive sense ($|\varphi| = 0.799$).

SAE	Feat	Fuzz	$ \varphi $	Description
Solo	2827	+0.727	—	<i>too</i> indicating excess or an undesirable degree; stronger when <i>too</i> emphasises a significant o
Joint	4741	+0.681	0.710	<i>too</i> indicating excess or an undesirable degree in a context of regret, limitation, or negative o
Joint	14213	+0.174	0.799	<i>too</i> concluding a sentence that extends or adds to a previous statement; stronger when <i>too</i> e

Table 3: **“let” pragmatic split.** Solo feature 17451 conflates three distinct pragmatic roles of *let*: direct suggestion/imperative, discourse-transition pivot, and permissive allowing. The joint SAE assigns each role to a distinct feature.

SAE	Feat	Fuzz	$ \varphi $	Description
Solo	17451	+0.641	—	<i>let</i> introducing permission, suggestion, or command; stronger in formal or imperative conte
Joint	7228	+0.646	0.692	<i>let</i> as direct suggestion or invitation (“Let’s go”, “Let me show you”); stronger with imperat
Joint	10152	+0.273	0.486	<i>let</i> as discourse-transition pivot (“Let me now turn to...”); stronger with definitive or emph
Joint	18650	+0.497	0.435	<i>let</i> as granting permission or allowing an action; stronger when the action has significant co

414 D Activation norm context

415 GPT-2 large residual stream at layer 20: mean $\|x\|_2 = 133.3$, std = 224.6 (heavy-tailed; std
 416 $>$ mean reflects occasional large-norm residuals). Solo SAE L2 = 0.097 is 0.073% of mean
 417 norm; the additional joint cost of 0.010 is 0.0075%. This framing clarifies that the +9.3%
 418 relative L2 increase, while non-negligible as a fraction of the SAE’s own reconstruction error,
 419 is essentially invisible relative to the magnitude of the underlying representations being
 420 encoded.

421 E Qualitative case-study tables

422 F Auto-interp pipeline details

423 Feature explanations were generated using the Delphi framework (Bills et al., 2023) with
 424 Qwen/Qwen2.5-72B-Instruct-AWQ served locally via vLLM in an isolated virtual envi-
 425 ronment to avoid dependency conflicts with the SAE training stack. For each feature, we
 426 collected activation contexts from a 5M-token pass and generated a natural-language de-
 427 scription. Fuzzing evaluation replaced the highest-activating tokens in each example with
 428 random tokens drawn from the same marginal distribution and asked the model to identify
 429 which example was real, scoring $+1/n_{\text{rounds}}$ per correct identification. Hungarian matching
 430 used cosine similarity of decoder columns (W_{dec}) to align joint and solo dictionaries; 20,456
 431 of 20,480 features received a match meeting the cosine threshold used for inclusion.

Table 4: **“cookies” sense split (Gemma 2 9B)**. Solo feature 52496 conflates the food sense (*baked cookies*) with the web-technology sense (*browser cookies, data tracking*). The joint SAE separates them: feat. 7892 inherits the food sense ($|\varphi| = 0.741$), while feats. 29453 and 34623 each specialize on the web/privacy sense ($|\varphi| = 0.875$ and 0.621 respectively).

SAE	Feat	Fuzz	$ \varphi $	Description
Solo	52496	+0.493	—	<i>cookies</i> in food/recipe or web-technology contexts; description conflates both senses without
Joint	7892	+0.548	0.741	<i>cookie</i> in food and baking contexts; stronger for specific recipe or food-product references.
Joint	29453	+0.345	0.875	<i>cookies</i> in web privacy, data-tracking, and browser-storage contexts; weaker for food mention
Joint	34623	+0.541	0.621	<i>cookies</i> in web-technology contexts emphasising data privacy and user-tracking mechanisms

Table 5: **“flash” register split (Gemma 2 9B)**. Solo feature 3387 conflates the general sense of a sudden, brief occurrence with the proper-noun sense of Adobe Flash software. The joint SAE separates them: feat. 63365 retains the sudden-event sense ($|\varphi| = 0.857$), while feat. 34102 isolates the Adobe Flash sense ($|\varphi| = 0.173$). Despite its low $|\varphi|$ —reflecting how rarely this sense fires—feat. 34102 achieves the highest fuzzing score in the Gemma analysis (+0.725), illustrating how the decomposability penalty can liberate a rare but highly coherent sense that the solo SAE submerges.

SAE	Feat	Fuzz	$ \varphi $	Description
Solo	3387	+0.602	—	<i>flash</i> as a sudden, brief appearance or occurrence; stronger for more intense or vivid context
Joint	63365	+0.363	0.857	<i>flash</i> as a sudden visual or physical phenomenon; stronger for unexpected or dramatic occur
Joint	34102	+0.725	0.173	<i>Flash</i> as web software and browser plugin (Adobe Flash); stronger in technical or historical